DIM0501 — Boas Práticas de Programação

# Relatório do Projeto da Unidade 1 — AgendaFácil

Prof. Marcus Terra · DIMAp/UFRN · Semestre 2026.1 · Deposito: 31/10/2026

| Integrantes (nome e matrícula) | Linguagem |
|---|---|
| Danilo Araújo de Moraes – 20250067023 | Python |
| **Repositório ou arquivo entregue** | |
| https://github.com/moraesadanilo/AgendaFacil.git | |

## 1. Diagnóstico do código-base

| # | Categoria | Anomalia e onde está | Por que é um problema |
|---|---|---|---|
| 1 | Manutenibilidade | Todas as funções – Sem nomenclatura semântica e realizando múltiplas funções. | O código não comunica para quem ler o que faz e como funciona. |
| 2 | Organização e estética | Uso de “números mágicos” no código da função add. | A regras de negócio fica ocultas e se o horário da clínica mudar será necessário ler todo código para revisar e alterar a informação. |
| 3 | Adequação e lógica | Função add exige `dur > 0` e `h < 18`, mas não limita tempo de duração e consequentemente o término do atendimento. | Falha da regra de negócio atendimentos terminando depois do expediente, gerando reclamação de mal funcionamento. |
| 4 | Programação defensiva e segurança | Todas as funções – Sem validação de formato de data e tipos dos argumentos de entrada. | O sistema aceita entradas inválidas, o que pode gerar várias falhas de lógica. |
| 5 | Programação defensiva e segurança | Try/except na função add | Erros são engolidos silenciosamente e falhas ou alertar não são comunicados ao usuário. Quem faz a manutenção tem que sair em busca da falha. |
| 6 | Manutenibilidade | Informações dos atendimentos salvos em uma lista acessada por posição. | Não saber intuitivamente o que está salvo em cada índice; Alteração de um campo desloca os índices e quebra as funções sem nenhum erro visível |

| | | | |
|---|---|---|---|
| 7 | Organização e estética | Na função lst, tem o comentário “lista os atendimentos do mês”, mas a função filtra um único dia. | O comentário falso induz ao erro durante a leitura. |
| 8 | Organização e estética | Comentários de ruído e código morto: TODO antigo não implementado. | Poluição visual que gera dúvida sobre se realmente é uma pendencia em aberto/vencida ou se houve decisão de não implementar. |
| 9 | Desempenho e recursos | Funções add e rem com vários laços aninhados. | Excesso de iterações redundantes desperdiça recursos de memória e processamento quando sistema escala para muitos agendamentos. |
| 10 | Desempenho e recursos | flag booleana for(...): ok = False) na função add. | O laço continua iterando por toda a agenda mesmo após já ter encontrado um conflito. |
| 11 | Adequação e lógica | A checagem de conflitos na função add compara apenas a hora inicial inteira ignorando a duração (dur) e os minutos. | Desperdiça o tempo útil da clínica. |

## 2. O que foi resolvido e o que ficou como dívida

| # | Situação | Como foi resolvida, ou por que ficou |
|---|---|---|
| 1 | Resolvida | Fatoração com separação de responsabilidades para funções de validação; nomeação e renomeação aplicação da PEP 8 com nomenclatura semântica. |
| 2 | Resolvida | Substituição de valores mágicos por constantes documentadas no topo do arquivo. |
| 3 | Resolvida | Implementação das funções de guarda para validações das regras de negócio. |
| 4 | Resolvida | Criação de validações de entrada chamadas como cláusulas de guarda no início das funções públicas. |
| 5 | Resolvida | Aplicação do conceito de Fail Fast. O bloco try/except foi removido e a lógica propaga as exceções customizadas, ficando a cargo do modulo interface o tratamento e comunicação ao usuário. |
| 6 | Resolvida | Substituição da lista aninhada por uma estrutura @dataclass Atendimento, com campos nomeados. |
| 7 | Resolvida | Comentários enganoso foi substituído por uma docstring no padrão Google. |
| 8 | Resolvida | O comentário TODO vencido foi apagado e função implementada. |
| 9 | Resolvida | Unificação de lógicas de checagem funções com responsabilidade única chamadas em sequência eliminando os laços aninhados. |

| | | |
|---|---|---|
| 10 | Resolvida | A flag booleana de controle foi removida e o retorno da validação se dá por raise assim que detecta conflito, interrompendo o processamento redundante imediatamente. |
| 11 | Dívida assumida | A conversão interna para minutos na função _verificar_disponibilidade resolveu a falha de sobreposição em consultas porem o desperdício de vagas fracionadas continua. Assume-se essa dívida técnica para não alterar a assinatura original das funções públicas. |

## 3. Decisões de projeto

| Decisão tomada | Alternativa descartada | Por que decidimos assim |
|---|---|---|
| Arquitetura módulos separados | Implementar prints e try/excepts dentro do módulo principal (main). | O modulo de execução (principal) ficaria puído poluído e seria necessário a implementação de um try/excepts para cada chamada de função. |
| A conversão das horas para minutos para simplificar a checagem de sobreposição de intervalos. | Converter hora em fração ou checar quantidades de vagas por hora ou alterar os parâmetros de entrada. | A primeira opção demandou maior complexidade de implementação; a segunda só foi pensada após conclusão dos módulos e ficou como dívida técnica. |
| Implementar a função cancelar_agendamento() com cancelamento por status lógico mantendo o registro na lista. | Remover o registro completamente da lista como sugerido em ag.remove(...) | Mantém o rastreamento dos agendamentos para fins estatísticos e de relatórios futuros. |
| Alteração das strings de impressão de sucesso do caminho feliz e textos de ERRO. | Manter exatamente as strings literais do código-base. | Deixar a comunicação com o usuário mais clara, informativa e padronizada, tanto para casos de sucesso como para as mensagens de erro geradas pelas exceções. |
| Criação de um script isolado para testes defensivos (testes_defensivos.py). | Adicionar as chamadas de teste de erro diretamente no módulo principal. | Para cumprir a exigência de manter a saída do programa mais próxima da versão original. |

## 4. Mudanças de comportamento

| Entrada ou situação | O que acontecia antes | O que acontece agora |
|---|---|---|
| Hora fora do expediente | Nada acontecia e nenhuma mensagem era exibida. | Interface exibe alerta de `EntradaInvalidaError` |
| Duração inválida | Ignorada em silêncio. | Interface exibe alerta de `EntradaInvalidaError` |

| | | |
|---|---|---|
| Atendimento que terminava após o expediente. | Era gravado. | É recusado com exibe alerta de `EntradaInvalidaError` |
| Data fora do formato AAAA-MM-DD ou inexistente | Era aceita. | Validada e recusada. |
| Sobreposição parcial de horários devido duração longa. | Era aceito pois a mesma hora de início era conflitava. | Sobreposição de intervalos exibe alerta `ConflitoDeHorarioError`. |

## 5. Como executar e gerar a documentação

| Item | Resposta |
|---|---|
| Comando para compilar e executar | python agenda_app.py |
| Comando para gerar a documentação de API | python -m pdoc agendamento interface -o docs<br>--docformat google |
| Onde a documentação gerada ficou no repositório | Pasta `docs/`; abrir `docs/index.html` no navegador. |
| Saída do programa de demonstração (cole aqui) | Atendimento marcado com sucesso para Ana<br>Operação recusada: Horário ocupado! Conflito com atendimento agendado às 9h com duração de 30 min.<br>Atendimento marcado com sucesso para Carla<br>Atendimento para Ana remarcado com sucesso<br><br>--- AGENDA DO DIA 2026-10-05 ---<br>14h - Paciente: Carla - Profissional: Dra. Souza |

## 6. Uso de ferramentas de IA

| Pergunta | Resposta |
|---|---|
| Foram usadas ferramentas de IA neste trabalho? | (  ) Não  ( X ) Sim |
| Em quais etapas, e para quê? | Apoio ao aprendizado da linguagem python, elaboração da função de validação do formato da data; identificação de erros (horário cancelado que continuava bloqueando a agenda) e tratamento de exceções; elaboração dos casos testes, revisão do código com sugestão da separação entre regras e interface (prints estavam nas funções e na main); elaboração da lógica do modulo interface; revisão gramatical e dos elementos padronizados no docstrings e rascunho do README. |